## Цель работы

Изучить структуру растровых графических файлов и способы хранения в них сведений об изображении. Разработать приложение на Python, которое извлекает основные метаданные из файлов JPG/JPEG, GIF, TIFF, BMP, PNG и PCX, отображает результаты в графическом интерфейсе и корректно обрабатывает ошибки чтения.

## Задачи

1. Изучить устройство заголовков и основных структур форматов JPG/JPEG, GIF, TIFF, BMP, PNG и PCX.
2. Реализовать ручное чтение байтов файлов и извлечение имени, размеров изображения, разрешения, глубины цвета и типа сжатия.
3. Реализовать определение фактического формата по сигнатуре файла, а не только по его расширению.
4. Создать графический интерфейс для выбора папки, просмотра результатов в таблице, индикации прогресса и отмены сканирования.
5. Организовать многопоточную обработку файлов, чтобы интерфейс оставался отзывчивым при сканировании больших папок.
6. Реализовать потоковое чтение заголовков и нужных структур без загрузки всего изображения в память.
7. Добавить обработку некорректных и повреждённых файлов и экспорт результатов в CSV.
8. Провести функциональную проверку обработки поддерживаемых форматов и описать ограничения реализованных проверок.

## Использованные средства разработки

Язык программирования — Python 3. Приложение использует Tkinter и ttk для построения настольного графического интерфейса, os и os.path для работы с каталогами и путями, struct для интерпретации байтовых полей, threading, queue и concurrent.futures.ThreadPoolExecutor для фоновой обработки, csv для экспорта результатов.

Сторонние графические библиотеки (например, Pillow) для извлечения метаданных не применяются. Данные читаются непосредственно из файлов в бинарном режиме. Для упаковки приложения в исполняемый файл Windows предусмотрен PyInstaller; он используется только на этапе сборки и не участвует в разборе форматов.

## Ход работы

### 1. Изучение структуры графических форматов

Для каждого формата определены сигнатура или начальные признаки, а также структуры, в которых хранятся геометрия изображения, глубина цвета, разрешение и сведения о сжатии.

Форматы различаются способом представления полей: некоторые используют фиксированный заголовок, другие — последовательность сегментов или чанков, а TIFF — каталог тегов.

### 2. Реализация чтения метаданных

В модуле image_metadata.py реализованы отдельные функции разбора форматов. Файлы открываются с режимом rb. Для чтения полей заданной длины используется вспомогательная функция, которая проверяет, что запрошенное количество байтов действительно получено. Числовые значения преобразуются с помощью struct с указанием порядка байтов. Если данных недостаточно или структура некорректна, создаётся ошибка разбора, которая преобразуется в понятный статус строки таблицы.

#### 2.1. Формат BMP

В BMP проверяется сигнатура BM. Затем читаются поля заголовка файла и размер DIB-заголовка. Из заголовка извлекаются ширина, высота, число бит на пиксель, количество плоскостей, код сжатия, смещение пиксельных данных и значения разрешения в пикселях на метр. Значения пикселей на метр при наличии переводятся в dpi умножением на 0,0254.

Для BMP реализовано отображение распространённых кодов сжатия: BI_RGB, BI_RLE8, BI_RLE4, BI_BITFIELDS, BI_JPEG, BI_PNG и BI_ALPHABITFIELDS. Если изображение индексированное, палитра представляет собой таблицу цветов, а значения пикселей указывают на элементы этой таблицы. Для глубины до 8 бит размер палитры оценивается по числу цветов в заголовке или по максимально возможному количеству индексов.

#### 2.2. Формат PNG

PNG начинается с сигнатуры 89 50 4E 47 0D 0A 1A 0A. Первым ожидается чанк IHDR длиной 13 байт. В нём читаются ширина, высота, глубина цвета, тип цвета, метод сжатия, метод фильтрации и режим чересстрочной развёртки. Длины чанков позволяют переходить к следующим структурам, не загружая всё содержимое в память.

При сканировании структуры проверяется, что длина чанка не выходит за размер файла, и ищется завершающий чанк IEND. Чанк pHYs может содержать плотность пикселей по горизонтали и вертикали. Если единица измерения равна метру, значения переводятся в dpi. Если физическое разрешение в файле не задано, интерфейс показывает «Не указано».

#### 2.3. Формат JPEG

JPEG анализируется как последовательность маркеров и сегментов. В начале проверяется маркер SOI FF D8. Маркеры SOF содержат точность компонента, высоту, ширину и количество компонентов изображения. Сегмент JFIF APP0 может содержать единицы и значения плотности. В зависимости от единицы разрешение интерпретируется как dpi или точек на сантиметр и переводится в dpi.

После разбора заголовков проверяется наличие завершающего маркера EOI FF D9 в конце файла. Поскольку JPEG может иметь разные режимы кодирования, в столбце сжатия выводится пояснение о типичном использовании DCT и определяется точность на компоненту по маркеру кадра.

### 2.4. Формат GIF

Для GIF проверяются сигнатуры GIF87a и GIF89a. Из Logical Screen Descriptor считываются ширина, высота и packed field, по которому определяется наличие глобальной таблицы цветов и её размер. GIF обычно использует сжатие LZW. В конце файла ожидается trailer — байт 3B; его отсутствие считается признаком повреждения.

### 2.5. Формат TIFF

В TIFF первые два байта задают порядок байтов: II для little-endian и MM для big-endian. Далее проверяется идентификатор 42 и считывается смещение первой директории IFD. IFD содержит число записей, а каждая запись включает тег, тип данных, количество значений и значение либо смещение к нему.

Реализовано чтение тегов ImageWidth (256), ImageLength (257), BitsPerSample (258), Compression (259), XResolution (282), YResolution (283), SamplesPerPixel (277) и ResolutionUnit (296). Значения разрешения преобразуются в dpi в зависимости от единицы измерения. Поддерживаются распространённые коды сжатия, включая отсутствие сжатия, CCITT, LZW, JPEG, Deflate и PackBits. В текущей версии разбирается первая IFD.

### 2.6. Формат PCX

PCX имеет заголовок фиксированной длины 128 байт. Проверяются manufacturer, версия и способ кодирования. Из заголовка считываются координаты границ изображения, разрешение по горизонтали и вертикали, число бит на плоскость, число цветовых плоскостей и число байтов на строку. Размеры вычисляются как xmax − xmin + 1 и ymax − ymin + 1. Для кодирования указывается RLE или отсутствие сжатия.

## 3. Интерпретация отображаемых характеристик

| Характеристика | Что означает | Откуда берётся |
|---|---|---|
| Имя файла | Название файла в выбранной папке. | Имя в файловой системе. |
| Размер изображения | Ширина × высота в пикселях. | Поля заголовка, сегменты JPEG или теги TIFF. |
| Разрешение | Плотность пикселей для печати/физического размера; может отсутствовать. | Поля BMP, JFIF, pHYs PNG, заголовок PCX или теги TIFF. |
| Глубина цвета | Число бит на пиксель, компоненту или индекс палитры — в зависимости от формата. | Поле bit depth, bits-per-pixel, BitsPerSample и число плоскостей. |
| Сжатие | Метод кодирования графических данных. | Код сжатия BMP/TIFF, метод GIF/PCX либо особенности JPEG/PNG. |

| | | |
|---|---|---|
| Размер файла | Объём файла на диске в байтах, КиБ, МиБ или ГиБ. | Размер файловой записи. |
| Статус | Результат чтения и базовых проверок. | Проверка сигнатуры, длины заголовков, смещений и завершающих маркеров. |

Важно: значение «глубина цвета» не полностью одинаково трактуется во всех форматах. Например, BMP обычно указывает число бит на пиксель, JPEG — точность на компоненту, а PNG — глубину одного канала либо индексированной палитры. Поэтому одинаковые числа в таблице не всегда означают одинаковую структуру хранения.

### 4. Реализация пользовательского интерфейса

Интерфейс создан на Tkinter/ttk. Пользователь выбирает каталог, включает при необходимости рекурсивный обход вложенных папок и запускает сканирование. Результаты отображаются в таблице с колонками: имя файла, формат, размеры, разрешение, глубина цвета, сжатие, размер файла и статус. Цветовое выделение помогает визуально отличать ошибки и предупреждения.

В строке состояния отображается количество найденных и обработанных файлов. Для остановки предусмотрена кнопка отмены. При выборе строки показывается полный путь и дополнительные сведения. Экспорт CSV сохраняет результаты с заголовками столбцов и кодировкой UTF-8 с BOM, что упрощает открытие файла в Excel.

### 5. Проверка ошибочных файлов

Перед разбором содержимого программа определяет формат по сигнатуре. Если сигнатура неизвестна или не соответствует поддерживаемому формату, файл не передаётся неподходящему парсеру. При недостатке данных, некорректной длине сегмента, недопустимом размере, смещении за пределы файла или отсутствии характерного завершающего маркера создаётся статус ошибки вместо аварийного завершения приложения.

Для PNG проверяется наличие IEND и корректность границ чанков; для JPEG — маркер EOI; для GIF — завершающий байт 3B. Для BMP проверяются основные размеры и смещение пиксельных данных; для TIFF — корректность первой IFD и границ тегов; для PCX — сигнатура, размеры и базовые поля заголовка. Эти проверки обнаруживают часть повреждений, но не гарантируют, что все пиксельные данные могут быть полностью декодированы.

### 6. Тестирование

Для проверки следует использовать тестовые изображения всех шести форматов, а также заведомо некорректные файлы. Проверяются запуск приложения, выбор папки, отображение результатов, прогресс, отмена сканирования и экспорт CSV.

| Тест | Ожидаемый результат | Результат проверки |
|---|---|---|
| Корректный файл PNG/BMP/JPEG/GIF/TIFF/PCX | Определяются формат и доступные метаданные. | Заполнить после запуска на тестовом наборе. |
| Текстовый файл с | Выводится сообщение о | Заполнить после проверки. |

| | | |
|---|---|---|
| расширением .jpg | неизвестной сигнатуре, приложение продолжает работу. | |
| Файл с обрезанным заголовком | Показывается статус повреждения/ошибки чтения. | Заполнить после проверки. |
| PNG без IEND или JPEG без EOI | Показывается предупреждение о повреждении. | Заполнить после проверки. |
| Каталог с большим количеством файлов | Окно остаётся отзывчивым, прогресс обновляется. | Заполнить после проверки. |
| Экспорт в CSV | Создаётся CSV с результатами и заголовками столбцов. | Заполнить после проверки. |

## Вывод

В ходе лабораторной работы разработано настольное приложение на Python для чтения метаданных растровых изображений JPG/JPEG, GIF, TIFF, BMP, PNG и PCX. Реализован ручной разбор заголовков, сегментов, чанков и тегов без сторонних библиотек для извлечения метаданных. Приложение отображает геометрию изображения, разрешение, глубину цвета, тип сжатия, размер файла и статус проверки.

Для удобства пользователя реализованы табличный интерфейс, выбор папки, обход вложенных каталогов, многопоточная обработка, прогресс, отмена сканирования и экспорт CSV. Добавлены проверки сигнатур и ряда структурных признаков повреждения. Таким образом, выполнены основные задачи лабораторной работы; при дальнейшей доработке можно расширить проверку целостности JPEG/PNG и добавить обход всех IFD многостраничных TIFF.